ENGINEERING BRIEF · AGENT H · NEW ONBOARDING

Roomio — Photo Texture Mapping

Let users texture their furniture from real photos. Detect the surface texture of an object in a user's photo, turn it into a tiling PBR material, and apply it to the matching Roomio archetype — so their own sofa's fabric or table's wood wraps onto the 3D asset. Every asset must support textures.

PHOTO → PBR TEXTURE | UV-MAPPED ASSETS | DEPENDS ON B's DETECTION

Repo: `~/Desktop/personal/roomio` · **Your folder:** `/texturing`
Coordination: `/shared` + `roomio.txt` + `LEARNINGS.md` · **Agent D merges**
Date: June 29, 2026 · Builds on B's detection + needs UV-mapped assets

Contents

1 Mission & Two Hard Dependencies 03

2 The Pipeline (photo → applied texture) 04

3 Stage 1 — Detect & Crop the Texture 05

4 Stage 2 — Make It a Tiling PBR Material 06

5 Stage 3 — Apply to the Asset (UVs) 08

6 "Every Asset Supports Textures" — UVs 09

7 Coordination & Schema 10

8 Requirements & Acceptance 11

9 Milestones 12

1. Mission & Two Hard Dependencies

> You are **Agent H**, joining the Roomio build (A interiors, B detection/camera, C house+persistence, D coordinator, E lighting, F QA/dedup, G rendering). Your mission: a user uploads a photo of their furniture; you detect the object's *surface texture* (the fabric weave, the wood grain), turn it into a usable tiling material, and apply it to the matching Roomio archetype so the 3D piece looks like *theirs*. First: read `roomio.txt` + `LEARNINGS.md`, post an onboarding entry.

TWO DEPENDENCIES THAT GATE EVERYTHING — CONFIRM THEM FIRST

(1) **Detection (Agent B).** Finding and cropping the object's surface region in a photo is exactly B's furniture-detection pipeline. Reuse it — don't rebuild object detection. Request a cropped texture region from B's output.
(2) **UV maps on assets.** A texture can only wrap onto a model that has UV coordinates (the 2D layout saying where the image maps onto the 3D surface). The documented failure mode is assets with *no uv attribute at all* — then a texture shows as garbage/dots. So "every asset supports textures" literally means "every asset must be UV-mapped" — that's a precondition you must guarantee (§6).

2. The Pipeline (photo → applied texture)

Three decoupled stages. Input: a user photo (+ which archetype it's for). Output: the archetype rendered with a material derived from the photo.

#	Stage	What it does
T1	Detect & crop	Locate the object in the photo (via B) and crop a clean, flat-ish patch of its surface texture (the fabric/wood/material region, not the whole object with its lighting and edges).
T2	Texture-ify	Turn that crop into a usable material: make it tile seamlessly, neutralize baked-in lighting, and derive PBR maps (albedo + roughness + normal) so it responds to light rather than looking like a flat sticker.
T3	Apply via UVs	Build a PBR material from the maps and assign it to the target archetype's material slot; it wraps correctly because the asset is UV-mapped. User confirms/adjusts scale.

SUGGESTION, WITH CONFIRMATION — LIKE B'S DETECTION
Mirror the rest of Roomio: the detected texture is a *proposal* the user previews and accepts/adjusts (scale, rotation, tighter crop), never auto-applied irreversibly. They can always revert to the archetype's default material.

3. Stage 1 — Detect & Crop the Texture

- **Reuse B's detection** to locate the furniture object + its category in the photo (so you know which archetype it maps to). Don't reimplement object detection.
- **Crop a surface patch,** not the whole object. You want a representative, relatively flat region of the material (a section of the sofa cushion face, the table top) — avoid edges, seams, deep shadows, and highlights that would poison the texture.
- **Perspective considerations:** a photo of a surface at an angle gives a foreshortened patch. Where feasible, take the most front-facing region; advanced handling can rectify perspective, but a good central crop is the pragmatic v1.
- This stage produces a clean RGB patch + the target archetype id. Hand-off from B should give you the bounding region; you refine to a clean patch.

GARBAGE IN, GARBAGE OUT

Texture quality is bounded by crop quality. A patch full of folds, shadow gradients, or the object's silhouette will tile badly. Invest in choosing/auto-selecting a clean, evenly-lit, flat region — it matters more than any later step.

4. Stage 2 — Make It a Tiling PBR Material

A raw photo crop applied directly looks wrong: it has visible seams when tiled, baked-in lighting, and no surface response. Three sub-steps fix that.

4a — Seamless tiling

- Furniture surfaces repeat (fabric weave, wood grain), so the texture must tile without a visible seam. Use offset-and-blend (wrap the patch, blend the seam line) or a frequency-domain/synthesis approach for irregular materials.
- Set the Three.js texture to `RepeatWrapping` on S and T so it tiles across the asset's UVs.

4b — De-light (neutralize baked lighting)

- A photo has the room's lighting baked in (bright spots, shadows). Applied as albedo, that fights the scene's lighting and looks fake. Flatten large-scale brightness variation (high-pass / illumination estimation) so the albedo is the material's true color, and the scene's lights (E/G) do the lighting.

4c — Derive PBR maps

- **Albedo** (base color) = the de-lit, tiled crop.
- **Roughness** = derived from the texture (e.g. inverse luminance/contrast heuristic) so fabric reads matte and polished wood reads slightly glossy.
- **Normal map** = derived from the albedo's luminance (height→normal) so weave/grain catches light with subtle relief — this is what makes it read as real surface, not a printed sticker.
- Assemble into a `MeshStandardMaterial` ({map, roughnessMap, normalMap}) with correct color space (albedo sRGB; roughness/normal linear). This dovetails with Agent G's PBR/IBL realism.

```
// T2/T3 — build a tiling PBR material from derived maps
const albedo = loader.load(albedoUrl); albedo.colorSpace = THREE.SRGBColorSpace;
const rough  = loader.load(roughUrl);  const normal = loader.load(normalUrl);
[albedo, rough, normal].forEach(t => { t.wrapS = t.wrapT = THREE.RepeatWrapping; t.repeat.set(tileX, tileY); })
const mat = new THREE.MeshStandardMaterial({ map: albedo, roughnessMap: rough, normalMap: normal });
targetMesh.material = mat;   // wraps correctly IF the mesh has UVs (§6)
```

5. Stage 3 — Apply to the Asset (UVs)

- Assign the derived PBR material to the correct material slot of the target archetype (a sofa may have separate fabric vs wood-leg slots — apply the fabric texture to the fabric slot only). Coordinate slot conventions with A/G.
- **User scale/rotation control:** expose `texture.repeat` (tiling density) and `texture.rotation` so the user can size the weave/grain to look right on their piece. This is the main adjustment users will want.
- **Preview + confirm + revert:** show it applied, let them accept, re-crop, or revert to default. Persist the chosen texture as part of the design (coordinate with C's persistence — store a reference/handle, not a giant embedded image where avoidable).

IT ONLY WRAPS RIGHT IF UVS ARE GOOD

Even a perfect texture looks broken on a model with missing or bad UVs (stretching, dots, mismatched scale). Stage 3 success is gated entirely by §6. Treat UV coverage as the foundation, not an afterthought.

6. "Every Asset Supports Textures" — UVs

> The brief's requirement that **all assets support texture** reduces to a concrete technical guarantee: **every archetype must have valid UV coordinates.** A texture cannot map onto a mesh with no UVs. This is the precondition the whole feature stands on.

What to do

- **Audit the asset corpus** (coordinate with A, who owns the library) for which archetypes have UVs and which don't. Catalogue the gaps in `roomio.txt`.
- **Assets with good UVs:** ready — textures apply directly.
- **Assets missing UVs:** generate them. For furniture, box/triplanar projection or auto-unwrap gives usable UVs; for cleanest results the asset should be unwrapped at authoring time. Where you must auto-generate, a triplanar/box-projection approach avoids the "only dots" failure for boxy furniture.
- **Establish a standard:** every new asset entering the library must ship with UVs + named material slots. Record this as a rule in `LEARNINGS.md` so A and any asset source comply going forward.

TRIPLANAR AS THE SAFETY NET

For furniture where proper unwrapping isn't available, triplanar/box projection (project the texture from three axes and blend) gives acceptable results without per-asset UV work — a pragmatic fallback so "every asset supports textures" holds even for un-unwrapped meshes. Prefer authored UVs where they exist; use triplanar as the floor.

7. Coordination & Schema

Stack

Need	Tool	Note
Object detect/crop	Agent B's detection pipeline	Reuse; request a cropped region for the target object.
Image processing	Canvas / OpenCV.js / WebGL shaders	Seamless tiling, de-light, height→normal derivation.
Material	THREE.MeshStandardMaterial (+ RepeatWrapping)	map + roughnessMap + normalMap; correct color space.
UV generation	Auto-unwrap / triplanar projection	Fallback for assets lacking UVs.

Schema (publish to /shared)

```
{
  "texture_job": { "source_photo": "ref", "target_archetype_id": "sofa_3_seater",
                   "target_slot": "fabric" },
  "result": {
    "albedo": "ref", "roughness": "ref", "normal": "ref",
    "tile": {"x":2,"y":2}, "rotation": 0,
    "status": "ok|needs_uv|low_quality"
  }
}
```

Coordination

- Code in `/texturing`; only other writable places `/shared`, `roomio.txt`, `LEARNINGS.md`. Commit own paths; **D merges**.
- Read `roomio.txt` + `LEARNINGS.md` each cycle. Watch `[USER -> AGENT-H]` (top priority) and `ISSUE -> AGENT-H` from D.
- Post `REQUEST -> AGENT-B` (cropped texture region for the detected object), `REQUEST -> AGENT-A` (asset UV audit + named material slots + the "all new assets ship with UVs" rule), `REQUEST -> AGENT-G` (PBR map conventions so textures and rendering agree), `REQUEST -> AGENT-C` (persist applied textures with the design).

8. Requirements & Acceptance

ID	Requirement	Stage
T-1	Detect + crop a clean surface patch from a user photo (reusing B), with the target archetype id.	T1
T-2	Crop tiles seamlessly (RepeatWrapping); visible seams removed.	T2
T-3	Baked-in lighting neutralized so albedo is true color.	T2
T-4	Derive PBR maps (albedo + roughness + normal) with correct color space.	T2
T-5	Apply as a PBR material to the correct archetype material slot.	T3
T-6	User can adjust tiling scale + rotation; preview; accept; revert to default.	T3
T-7	EVERY archetype has valid UVs (audit + generate/triplanar fallback); new-asset UV rule recorded.	UVs
T-8	Texture is a suggestion: previewed + confirmed, never auto-applied irreversibly.	UX
T-9	Applied textures persist with the design (via C), stored efficiently.	Persist
T-10	Code in /texturing; coordinate via /shared+roomio.txt+LEARNINGS.md; D merges; no edits to others' source.	Coord

Acceptance test

Upload a photo of a fabric sofa; the system (via B) locates it, crops a clean cushion-fabric patch, makes it tile seamlessly, neutralizes the photo's lighting, derives albedo+roughness+normal, and applies it to the 3-seater archetype's fabric slot — the 3D sofa now shows the user's fabric, wrapping correctly with no stretching or dots, responding to room lighting (G/E). The user nudges the tiling scale and rotation, previews, accepts; another photo can revert. Repeat with a wood texture on a table top. Confirm an archetype that previously lacked UVs gets them (or triplanar) so the texture applies cleanly. The applied texture survives save/reload (via C).

9. Milestones

Milestone	Deliverable	Definition of done
H0	Read roomio.txt + LEARNINGS.md; scaffold /texturing; confirm the two dependencies (B detection, asset UVs) with B + A; publish texture schema; onboarding entry	Dependencies confirmed; schema in /shared.
H1	UV audit of the asset corpus + triplanar fallback so every asset can receive a texture	Every archetype takes a test texture without dots/stretching.
H2	T1: detect + crop a clean surface patch (via B) → target archetype	A photo yields a clean material patch + archetype id.
H3	T2: seamless tiling + de-light + derive albedo/roughness/normal	Patch becomes a tiling PBR material that responds to light.
H4	T3: apply to slot + user scale/rotation + preview/accept/revert	User's fabric/wood wraps onto the asset, adjustable.
H5	Persistence (via C) + polish + new-asset UV rule in LEARNINGS.md	Textures survive save/reload; standard recorded.

One-line summary. In `/texturing` : take a user's furniture photo, detect+crop its surface (reusing Agent B), turn that crop into a seamless, de-lit PBR material (albedo+roughness+normal), and apply it to the matching archetype's material slot with user scale/rotation control — guaranteeing every asset is UV-mapped (audit + triplanar fallback) so "all assets support textures" holds. Suggestion-with-confirm; persists via C; coordinate via /shared + roomio.txt + LEARNINGS.md; D merges.

End of brief · Roomio Photo Texture Mapping (Agent H) · A texture only wraps if the asset has UVs — guarantee that first.